

Auto Parallel LLVM 22 版 仕様書

auto-parallel project

2026 年 9 月 1 日

目次

1	目的と適用範囲	2
2	システム構成	2
2.1	ディレクトリ構成	2
2.2	主要コンポーネント	2
3	パス実行方式	3
4	ディレクティブ表現	3
5	ビルド仕様	3
5.1	必要環境	3
5.2	パス本体のビルド	3
5.3	パス本体ビルドの内部処理	4
5.4	再ビルドと並列度	4
6	PolyBench 検証仕様	4
6.1	処理フロー	4
6.2	PolyBench コンパイルの詳細	5
6.3	全件コンパイル時の動作	6
6.4	個別プログラム	6
6.5	全 30 プログラム	6
6.6	対象ベンチマーク	7
7	生成物とログ	7
8	終了状態	7
9	環境変数	8
10	既知事項と将来拡張	8
11	推奨運用手順	8

1 目的と適用範囲

本書は、改造版 LLVM 8 に組み込まれていた自動並列化処理を LLVM 本体から分離し、LLVM 22 環境で外部モジュールとしてビルド・実行・検証するための現行仕様を示す。対象は `auto-parallel-passes` と、PolyBench/C 4.2 を利用する `polybench` の検証スクリプトである。

現段階では処理本体に Legacy Pass Manager (Legacy PM) を使用する。LLVM 22 標準の `opt-22` から Legacy PM を直接実行せず、専用ドライバを使用する。New Pass Manager (New PM) 用の `Plugin.cpp` は将来移行用として残すが、現在のビルド対象には含めない。

2 システム構成

2.1 ディレクトリ構成

パス	用途
<code>auto-parallel-passes/include/AutoParallel</code>	LLVM 22 向けヘッダー
<code>auto-parallel-passes/lib</code>	各解析・変換パスの実装
<code>auto-parallel-passes/tools/LegacyDriver.cpp</code>	Legacy PM 専用実行ドライバ
<code>auto-parallel-passes/legacy/llvm8</code>	抽出時の LLVM 8 参照実装。原則として編集しない
<code>auto-parallel-passes/build-llvm22</code>	CMake ビルド成果物
<code>polybench/polybench-c-4.2</code>	PolyBench/C 4.2 本体
<code>polybench/prog/llvm22</code>	ベンチマーク別 IR、実行ファイル、ログ、ダンプ

2.2 主要コンポーネント

コンポーネント	役割
<code>DependencyCheck</code>	依存関係解析
<code>ReductionDetect</code>	リダクション変数・演算の検出
<code>AllPrivateDetect</code>	private 化対象の解析
<code>DirectiveInsertion</code>	対象ループへ自動並列化マーカを挿入
<code>ParamGet</code>	マーカと解析結果から OpenMP ランタイム呼び出しを含む IR を生成
<code>AutoParallelLegacy.so</code>	上記 Legacy パスを含む外部共有ライブラリ

コンポーネント	役割
auto-parallel-legacy-driver	LLVM 22 上で Legacy パスを段階実行する専用コマンド

3 パス実行方式

専用ドライバは入力 LLVM IR を読み込み、Legacy PassManager へ対象の ModulePass を登録し、変換後 IR を出力する。

```
auto-parallel-legacy-driver -stage=directive -ditarget=kernel \
    input.ll -o directive.ll

auto-parallel-legacy-driver -stage=paramget -ditarget=kernel \
    directive.ll -o parallel.ll
```

-stage=directive は DirectiveInsertion、-stage=paramget は ParamGet を実行する。-ditarget=kernel は現行パス側のコマンドラインオプションであり、PolyBench の対象カーネル選択に使用する。

4 ディレクティブ表現

LLVM 8 本体へ追加されていた独自 `llvm.directive intrinsic` には依存しない。LLVM 22 版では通常の宣言関数 `__auto_parallel_directive()` をマーカーとして呼び出し、節情報を `!auto_parallel metadata` として Call 命令へ付加する。

この表現は `include/AutoParallel/DirectiveMarker.h` へ集約されている。将来、intrinsic や別形式へ戻す場合は、原則として同ヘッダーの生成・参照ヘルパーを変更境界とする。

5 ビルド仕様

5.1 必要環境

- LLVM/Clang 22 (clang-22, opt-22, LLVM CMake package)
- CMake 3.20 以上
- C++17 対応コンパイラ
- OpenMP ヘッダーおよび libomp

5.2 パス本体のビルド

```
cd ~/auto-parallel/auto-parallel-passes
cmake -S . -B build-llvm22 \
    -DLLVM_DIR=/usr/lib/llvm-22/lib/cmake/llvm \
    -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake --build build-llvm22 --parallel
```

主な成果物は次の 2 つである。

- build-llvm22/AutoParallelLegacy.so
- build-llvm22/auto-parallel-legacy-driver

5.3 パス本体ビルドの内部処理

最初の `cmake -S . -B build-llvm22` はコンパイルそのものではなく、LLVM 22 の構成を読み込み、ビルド規則を生成する処理である。`find_package(LLVM REQUIRED CONFIG)` は `LLVM_DIR` 以下の `LLVMConfig.cmake` を読み込み、ヘッダー位置、ライブラリ、コンパイル定義を取得する。CMake は検出した LLVM のメジャーバージョンが 22 以外の場合、構成段階で停止する。

`cmake --build` では、次のソースを C++17 としてコンパイルする。

```
lib/DependencyCheck.cpp
lib/ReductionDetect.cpp
lib/AllPrivateDetect.cpp
lib/DirectiveInsertion.cpp
lib/ParamGet.cpp
```

同じ処理本体から 2 種類の成果物を生成する。`AutoParallelLegacy.so` は外部ロード可能な MODULE ライブラリである。`auto-parallel-legacy-driver` は同じソースを直接リンクし、LLVM 22 で Legacy PM を確実に起動する実行ファイルである。現行の PolyBench スクリプトは共有ライブラリの存在も事前確認するが、実際の変換実行には専用ドライバを使用する。

5.4 再ビルドと並列度

ソース変更後は通常、構成をやり直さず次のコマンドだけでよい。

```
cmake --build ~/auto-parallel/auto-parallel-passes/build-llvm22 --parallel
```

CMake 設定または LLVM の場所を変更した場合は、`cmake -S/-B` による再構成も行う。メモリ不足になる環境では `--parallel 1` を指定する。

6 PolyBench 検証仕様

6.1 処理フロー

- 1 プログラムのコンパイルは次の順序で行う。
 1. `clang-22 -S -emit-llvm` で LLVM IR を生成する。
 2. `mem2reg`, `early-cse`, `loop-mssa(licm)` で IR を正規化する。
 3. 専用ドライバの `directive` 段を実行する。
 4. LLVM verifier でディレクティブ挿入後 IR を検証する。
 5. 専用ドライバの `paramget` 段を実行する。
 6. LLVM verifier で並列化後 IR を検証する。
 7. 正規化 IR から逐次版、並列化 IR から OpenMP 並列版を生成する。

コンパイル時には `POLYBENCH_TIME` と `POLYBENCH_DUMP_ARRAYS` を定義する。実行時は逐次版と並列版の配列ダンプを保存・画面表示し、バイト単位で比較する。

6.2 PolyBench コンパイルの詳細

`compile-llvm22.sh <benchmark>`は次の 5 段階を順番に実行する。途中のコマンドが非 0 で終了した場合は `set -e` により直ちに停止し、不完全な実行ファイルを検証工程へ渡さない。

6.2.1 第 1 段階：C ソースから LLVM IR を生成

対象ソースは PolyBench ツリーから `*/<benchmark>/<benchmark>.c` に一致する最初のファイルを検索する。見つからない場合は開始前に終了する。

```
clang-22 -S -emit-llvm -Xclang -disable-O0-optnone \  
-I polybench-c-4.2/utilities \  
-DPOLYBENCH_TIME -DPOLYBENCH_DUMP_ARRAYS \  
<benchmark>.c -o <benchmark>.ll
```

`-S -emit-llvm` は機械語ではなくテキスト LLVM IR を生成する。`-disable-O0-optnone` は、O0 生成関数へ `optnone` 属性が付いて後段の解析・変換を妨げることを防ぐ。`POLYBENCH_TIME` は実行時間、`POLYBENCH_DUMP_ARRAYS` は結果配列の出力を有効にする。

6.2.2 第 2 段階：解析しやすい IR へ正規化

```
opt-22 -passes='mem2reg,early-cse,loop-mssa(lcm)' -S \  
<benchmark>.ll -o <benchmark>.normalized.ll
```

`mem2reg` は局所変数の Load/Store を SSA 値と PHI へ昇格する。`early-cse` は明白な共通部分式と重複メモリアクセスを整理する。`loop-mssa(lcm)` は MemorySSA を利用してループ不変命令を整理する。この正規化により、独自パスが想定するループ、PHI、依存関係を解析しやすくする。

6.2.3 第 3 段階：並列化マーカーを挿入

```
auto-parallel-legacy-driver -stage=directive -ditarget=kernel \  
<benchmark>.normalized.ll -o <benchmark>.directive.ll \  
2> directive.log  
opt-22 -passes=verify -disable-output <benchmark>.directive.ll
```

`DirectiveInsertion` は対象ループを選び、通常関数呼び出しと metadata からなる自動並列化マーカーを挿入する。診断出力は `directive.log` へ保存する。直後の verifier は、型不一致、壊れた CFG、不正な PHI などを検出し、不正な IR が次段へ進むことを防ぐ。

6.2.4 第 4 段階：OpenMP 並列 IR を生成

```
auto-parallel-legacy-driver -stage=paramget -ditarget=kernel \  
<benchmark>.directive.ll -o <benchmark>.parallel.ll \  
2> paramget.log  
opt-22 -passes=verify -disable-output <benchmark>.parallel.ll
```

`ParamGet` はマーカー、ループ解析、依存関係、private/reduction 情報を利用し、アウトライン関数、スレッド ID、ループ境界、private 領域、reduction 処理、`__kmpc_*` ランタイム呼び出しを含む IR を構築する。診断は `paramget.log` へ保存し、生成 IR を再び verifier へ通す。

6.2.5 第 5 段階：逐次版と並列版をリンク

逐次版は正規化 IR、並列版は変換済み IR から生成する。両者に同じ PolyBench ランタイム utilities/polybench.c と数学ライブラリ-lm をリンクする。

```
clang-22 ... polybench.c <benchmark>.normalized.ll -lm \  
-o <benchmark>.sequential  
  
clang-22 ... -fopenmp -I <OMP_INCLUDE_DIR> -L <OMP_LIBRARY_DIR> \  
-Wl,-rpath,<OMP_LIBRARY_DIR> \  
polybench.c <benchmark>.parallel.ll -lm \  
-o <benchmark>.parallel
```

並列版の-fopenmp は OpenMP ランタイムとのリンクを有効にする。-I は OpenMP ヘッダー、-L は libomp の検索先、-Wl,-rpath は実行時の共有ライブラリ検索先を実行ファイルへ記録する。既定値はプロジェクト内の旧ビルド成果物であり、LLVM 22 版 OpenMP を別途用意した場合は OMP_INCLUDE_DIR と OMP_LIBRARY_DIR で上書きできる。

6.3 全件コンパイル時の動作

compile-all-llvm22.sh は benchmarks-llvm22.sh の 30 件を順番に処理し、各対象について compile-llvm22.sh を起動する。出力は画面へ表示すると同時に prog/llvm22/<benchmark>/compile.log へ保存する。

個別コンパイルが失敗しても次の対象へ進む。各対象の PASS/FAIL とログ位置を prog/llvm22/compile-summary.tsv へ記録し、1 件以上失敗した場合は全件処理終了後に非 0 で終了する。このため、コンパイルエラーの調査では最初に集計 TSV を確認し、次に対象の compile.log、directive.log、paramget.log の順に確認する。

6.4 個別プログラム

```
cd ~/auto-parallel/polybench  
./compile-llvm22.sh bicg  
OMP_NUM_THREADS=4 ./run-llvm22.sh bicg
```

一括互換コマンドは次のとおりである。

```
./verify-llvm22.sh bicg
```

6.5 全 30 プログラム

コンパイルと実行は明確に分離する。

```
./compile-all-llvm22.sh  
OMP_NUM_THREADS=4 ./run-all-llvm22.sh
```

各一括スクリプトは、1 件が失敗しても残りを継続する。最後に成功数・失敗数と TSV 形式の集計ファイルを表示する。全配列ダンプを画面に表示するため出力量は大きい。端末出力全体も保存する場合は次を使用する。

```
./run-all-llvm22.sh 2>&1 | tee all-polybench.log
```

6.6 対象ベンチマーク

対象一覧は `benchmarks-llvm22.sh` で一元管理する。

correlation	covariance	2mm	3mm
atax	bicg	doitgen	mvt
gemm	gemver	gesummv	symm
syr2k	syrk	trmm	cholesky
durbin	gramschmidt	lu	ludcmp
trisolv	deriche	floyd-warshall	nussinov
adi	fdtd-2d	heat-3d	jacobi-1d
jacobi-2d	seidel-2d		

7 生成物とログ

ベンチマーク名を<benchmark>とすると、主な生成物は次のとおりである。

ファイル	内容
<code>prog/llvm22/<benchmark>/<benchmark>.ll</code>	Clang 生成 IR
<code><benchmark>.normalized.ll</code>	正規化後 IR
<code><benchmark>.directive.ll</code>	マーカー挿入後 IR
<code><benchmark>.parallel.ll</code>	OpenMP 変換後 IR
<code><benchmark>.sequential</code>	逐次実行ファイル
<code><benchmark>.parallel</code>	並列実行ファイル
<code>directive.log, paramget.log</code>	各変換段の標準エラー
<code>sequential.dump, parallel.dump</code>	PolyBench 配列ダンプ
<code>sequential.time, parallel.time</code>	実行時間
<code>result.diff</code>	配列不一致時の unified diff
<code>compile.log, run.log</code>	全件処理時のベンチマーク別ログ

全件集計は `prog/llvm22/compile-summary.tsv` と `prog/llvm22/run-summary.tsv` へ保存する。

8 終了状態

終了コード	意味
0	コンパイル成功、または逐次版と並列版の結果が一致
1	入力・成果物不足、変換、IR 検証、コンパイル、実行などの一般エラー
2	逐次版と並列版の配列ダンプが不一致

終了コード	意味
その他	子プロセスの異常終了コード（例：シグナルによる終了）

全件スクリプトは各対象の終了状態を集計し、1 件以上失敗した場合に最終的に非 0 で終了する。

9 環境変数

変数	仕様
OMP_NUM_THREADS	並列実行のスレッド数。未指定時は 4
OMP_INCLUDE_DIR	OpenMP ヘッダー位置の上書き
OMP_LIBRARY_DIR	libomp 位置の上書き。リンク時の rpath にも設定

10 既知事項と将来拡張

- LLVM 22 では opaque pointer であるため、ポインタ型だけから要素型を復元できない。型が必要な IR 生成では、元命令、Alloca、Load/Store、関数シグネチャなど意味を保持する情報源から明示的に取得する。
- 現行の処理本体は Legacy PM で動作する。New PM 化では解析処理と解析結果を分離し、PassBuilder へ登録する必要がある。
- Plugin.cpp は New PM 移行候補として保持されるが、現行 CMake 対象外である。
- 既存の全件検証では covariance の結果不一致と gramschmidt の ParamGet 異常終了が観測されている。スクリプト変更中に中断された対象は、compile-all-llvm22.sh と run-all-llvm22.sh で再検証する。
- 性能値は実行環境、入力サイズ、スレッド数に依存する。本仕様の PASS 条件は速度向上ではなく、IR 検証成功と配列ダンプ一致である。

11 推奨運用手順

```
# 1. パスと専用ドライバをビルド
cd ~/auto-parallel/auto-parallel-passes
cmake --build build-llvm22 --parallel

# 2. 全ベンチマークをコンパイル
cd ~/auto-parallel/polybench
./compile-all-llvm22.sh

# 3. 全ベンチマークを実行・比較
OMP_NUM_THREADS=4 ./run-all-llvm22.sh 2>&1 | tee all-polybench.log

# 4. 集計を確認
column -ts $'\t' prog/llvm22/compile-summary.tsv
```

```
column -ts $'\t' prog/llvm22/run-summary.tsv
```